AeroMorse Usage Guide — Changing Your Device Safely

This guide is for the day-to-day job of **changing an AeroMorse that is already working**: adding a macro, remapping a pattern, changing a password or a setting. It explains what each helper tool does and gives a tested, step-by-step order that keeps a working device working.

> **Why the care?** For many AeroMorse users the device is their *only* way to use a computer. A single bad character in `morse_map.py` can stop it from starting. Every step below exists so that can't happen — and so that if it ever does, you can put the old files back in a minute.

The five rules

1. **Back up the device before every change.** One double-click, dated, see Back up my AeroMorse.
2. **Never edit files directly on the device.** Edit the copies in your work folder, check them, then copy them over.
3. **Copy nothing to the device until the validator says PASS.**
4. **Save as UTF-8, not "UTF-8-BOM"** (Notepad++: *Encoding* → *UTF-8*). A BOM stops the device; the validator catches it.
5. **Keep the last good backup until the new version has worked for a day.**

---

1. Set up your work folder (one time)

Create a folder: `Documents\AeroMorse`. This is your *work folder* — everything you edit and every tool you use lives here, and your backups go in a `Backups` folder inside it.

What goes in it

Put in the work folder	From	Why
`boot.py`, `code.py`, `config.py`, `morse_map.py`	**your device** (the CIRCUITPY drive)	The copies you edit. Take them from the device, not the repo, so your own settings and patterns are what you edit.
`macro_secrets.enc` *(or `macro_secrets.txt`)*	your device, if you already have one	Your passwords (see AeroMorse Secrets).
`macro_secrets.example.txt`	the repo	Template for your **first** secrets file — see First time.
`aeromorse_validator.exe` + `Check my AeroMorse files.bat`	the AeroMorse repo	Safety check before copying to the device.
`AeroMorse Secrets.exe` + `Edit my AeroMorse secrets.bat`	the repo	Edit and PIN-lock your passwords.
`Back up my AeroMorse.bat`	the repo	One-click dated backup of the device.
`morse_map_analyzer.exe`	the repo	Finds duplicate and free patterns.
`aeromorse_cheatsheet.htm`	the repo	Interactive cheat sheet of *your* map.
`AeroMorse Cheat Sheet.pdf`, `AeroMorse – Keycode Reference.pdf`, `AEROMORSE_BUILD_GUIDE.pdf`, this guide	the repo	Reference to read or print.

When finished it looks like this:

```
Documents\AeroMorse\
    boot.py  code.py  config.py  morse_map.py  macro_secrets.enc
    Back up my AeroMorse.bat
    Check my AeroMorse files.bat     aeromorse_validator.exe
    Edit my AeroMorse secrets.bat    AeroMorse Secrets.exe
    morse_map_analyzer.exe           aeromorse_cheatsheet.htm
    AeroMorse Cheat Sheet.pdf        (and the other PDFs)
    Backups\
        AeroMorse-Green-2026-09-26_1430\
        ...
```

Give each device a name

Put an empty text file on the device named **`AeroMorse-<Name>.txt`** — e.g. `AeroMorse-Green.txt`. It does nothing on the device, but the backup tool and AeroMorse Secrets use it to label that device, which matters

as soon as you have two.

---

2. The tools — what each one does

Back up my AeroMorse

`Back up my AeroMorse.bat` — double-click with the device plugged in. It copies *everything* on every AeroMorse drive it finds into `Backups\<name>-<date>_<time>\` (for example `Backups\AeroMorse-Green-2026-09-26_1430`). It only reads the device; nothing there is changed. Two devices plugged in = two backups in one click.

Use it: at the start of every change, before a CircuitPython update, and any time the device is working well and you want a known-good copy.

Validator — "Check my AeroMorse files"

`Check my AeroMorse files.bat` runs `aeromorse_validator.exe` on the files in the work folder and answers one question: *will these files load on the device, or lock me out?*

- Checks `boot.py`, `code.py`, `config.py`, `morse_map.py` and your secrets file for the things that stop the device: a BOM, a Python syntax error, an import error, letter-O/letter-l typed instead of zero/one in a pattern, a bad setting value.
- **PASS** (green) = safe to copy. **FAIL** (red) = fix the line it names first. **WARN** (yellow) = the device will still run, but look at it (for example a secret key with no value, or a pattern stored under the wrong length so it can never be used).
- **Hidden characters:** if a stray invisible character slipped in while typing (for example from Ctrl+E), it lists each one and asks "*Remove them now? Type Y then Enter*". It saves a backup (`<file>.before-fix-<time>.bak`) before it changes anything.
- It never prints a password — only key names and counts.

AeroMorse Secrets

`Edit my AeroMorse secrets.bat` opens `AeroMorse Secrets.exe` on the work folder. Passwords and personal details never go in `morse_map.py`; they live in a separate secrets file, and `morse_map.py` only *names* them. A line like

```
g3[4][0b0110] = _secret('password1')   # .--.  P
```

means: *pattern* `.--.` *in Group 3 types whatever is stored under the key* `password1` — or the placeholder text in brackets if there's no such key.

There are two kinds of secrets file:

File	What it is	On the device
`macro_secrets.txt`	Plain text, `key=value` lines. Anyone who opens the drive can read it.	Always available, no PIN.
`macro_secrets.enc`	The same lines, **encrypted** by AeroMorse Secrets with a PIN you choose.	Locked at every power-up until you type the PIN in Morse. **Recommended.**

First time: creating your secrets

You start from the template, **`macro_secrets.example.txt`**.

1. **Copy the template.** In your work folder, copy `macro_secrets.example.txt` and rename the copy to **`macro_secrets.txt`**. (Windows may hide the `.txt` ending — if the copy is shown as `macro_secrets.example - Copy`, rename it to just `macro_secrets`.)
2. **See which keys your map uses.** Open `morse_map.py` and search for `_secret(`. Each one names a key — in the standard map: `name`, `address`, `phone`, `email`, `password1`, `wifi` (Group 3 letters A B C E P W) and `bank_login` (Group 6 R). The template already has a line for each.
3. **Fill in your values.** Open `macro_secrets.txt` in Notepad++ and replace each example value with your real one — everything after the first `=`: `name=Jane Smith password1=My-Real-P@ssword`
   - One secret per line. No quotes needed; symbols such as `@ ! $ # =` are fine in a value.
   - The key (before `=`) must be spelled **exactly** as in `_secret('…')`.
   - Delete lines you don't need. Add a line for any key you add to `morse_map.py` later.
   - Lines starting with `#` are notes; you can delete them.
   - Save as **UTF-8** (Notepad++: *Encoding → UTF-8*).
4. **Encrypt it (recommended).** Double-click **Edit my AeroMorse secrets.bat**. AeroMorse Secrets opens and imports `macro_secrets.txt` automatically — the status line says "*Imported plain macro_secrets.txt*".
   - Check the lines look right.
   - Type a **PIN** in both boxes (tick **Show PIN** to see it). Use letters and digits you can type in Group 1; 8 or more is best; capitals don't matter.
   - Click **Save (encrypt)**. It tells you about any key your `morse_map.py` uses that still has no value, then offers to **delete the plain `macro_secrets.txt`** — say **Yes** (the encrypted file now holds everything).
5. **Put it on the device.** Click **Copy to device** and pick the device. It offers to remove any old plain-text secrets file from the device — say **Yes**. The device restarts; don't sip or puff for ~5 seconds.
6. **Try it.** Use a secret pattern (e.g. Group 3 `P`). The screen shows **ENTER PIN + ENTER**: type your PIN in Morse with the Group 1 letters/digits, then Enter (`.-.-`). It unlocks and types the secret.

No PIN wanted? Skip steps 4–5 and copy `macro_secrets.txt` itself to the device. It works the same, but anyone who plugs the device into a computer can read your passwords.

Later: changing a secret

1. Double-click **Edit my AeroMorse secrets.bat**. It asks for your PIN to open `macro_secrets.enc`.
2. Change, add or delete `key=value` lines.

3. **Save (encrypt)** — leave the PIN boxes empty to keep your current PIN, or type a new one twice to change it.
4. **Copy to device** — to every device that uses these secrets.

Forgot the PIN? Only the secrets are affected; keyboard and mouse keep working. Start again from the template (First time, step 1).

On the device

- Secrets start **locked** after every power-up. The first secret you use asks for the PIN; nothing you type while entering it reaches the computer.
- Group 3 `U` (`..-`) unlocks without typing anything; `L` (`.-..`) locks again. Esc cancels PIN entry.
- The screen and USB log only ever show `SECRET <key>`, never the value.

Cheat sheet

`aeromorse_cheatsheet.htm` — open it in your browser (double-click). If the patterns don't appear by themselves, click 📁 **Select morse_map.py** and pick the one in your work folder. It shows every pattern of *your* map, group by group, as dots and dashes; click a row to hear its timing; 🖨️ **Print Cheat Sheets** makes a paper copy (or *Microsoft Print to PDF*). Secrets show only their key name with a 🔒, never the value.

`AeroMorse Help.html` is the same cheat sheet as a look-it-up window: legend at the top, one tab per group, made from your map whenever the cheat sheet PDF is built. §7 shows how to open and close it with one Morse code.

Optional: `Build my cheat sheet.bat` (needs Python and a copy of the repo) prints `AeroMorse Cheat Sheet.pdf` from your map in one click.

Morse map analyzer

`morse_map_analyzer.exe` — double-click; it reads the `morse_map.py` next to it and writes `morse_map_report.txt` (open it in Notepad). The report lists:

1. **Duplicate patterns** in a group (the later one silently wins);
2. patterns that **clash with Group 0** (the always-on group-switch codes win);
3. **unused patterns** of each length in each group — where to put a new macro without disturbing anything.

Reference documents

- **Keycode Reference** (PDF) — every `Keycode.NAME` you can use in `morse_map.py`, and how to type shifted symbols.
- **Build Guide** (PDF) — hardware, wiring, every `config.py` setting (§10) and troubleshooting (§12).
- **README** — group-by-group pattern tables and the full secrets guide.
- **Switch Mode Guide** — Group 9, where a sip holds Enter and a puff holds Space like two plain switches (switch games such as Benny's Hub).

- **Switch Control Guide** — Group 4 with iOS Switch Control, Android Switch Access and Samsung Universal Switch.

---

3. The recommended order for any change

This is the order used when updating the AeroMorse devices this project was built on. Each step protects the next one.

#	Step	Tool	Why at this point
1	**Back up the device**	Back up my AeroMorse	A dated, known-good copy *before* anything changes.
2	**Refresh your work copy** of the file you'll edit from the device	File Explorer	So you edit what is really on the device, not an older copy. Skip if you're sure the work folder is current.
3	**Plan**: find a free pattern	Analyzer report, Keycode Reference	Avoids reusing a pattern that already does something.
4	**Edit** `morse_map.py` (or `config.py`)	Notepad++ (UTF-8)	Make the change.
5	**Re-run the analyzer**	Analyzer	Catches a duplicate or Group 0 clash you just created.
6	**Secrets** — only if you added or renamed a `_secret()` key	AeroMorse Secrets	Add the value, **Save (encrypt)**. Its key check reads the `morse_map.py` you just edited.
7	**Validate** — must say **PASS**	Check my AeroMorse files	The final gate: checks every file exactly as the device will load it. Fix and re-run until PASS.
8	**Copy to the device** only the file(s) you changed	File Explorer (secrets: *Copy to device*)	The device restarts by itself. **Don't sip or puff for ~5 seconds** while it recalibrates.
9	**Test on the device**	—	Try the new pattern; if you changed secrets, try one (PIN).
10	**Update the cheat sheet**	Cheat sheet / Build my cheat sheet	So the printed sheet matches the device.

If anything goes wrong after step 8, restore — see Putting a backup back.

Why the validator is *after* Secrets and the analyzer: it is the last word on whether the files are safe, so it should see the finished files. Running it earlier is fine too — just always run it again last.

Pop-ups on the computer when the device restarts

Each restart reconnects the device's `CIRCUITPY` drive, so Windows AutoPlay, your antivirus and cloud-backup tools such as Google Drive may each show a pop-up near the clock. They are harmless and can be ignored, or switched off in each program — see **Troubleshooting → Pop-ups on the computer when the device starts or restarts** in the README / Build Guide for the steps.

4. Changing `config.py`

Same order as above (skip steps 3, 5, 6 and 10). Things to know:

- Settings are explained in the file itself and in **Build Guide §10**.
- `config.py` is **per device**. Two devices can need different sensor settings (for example a different puff threshold). Before editing, copy *that* device's `config.py` into the work folder (step 2), and copy it back only to that device.
- On/off settings accept `True` / `False` in any case; don't put them in quotes.

---

5. Putting a backup back

1. Open `Backups\` and pick the newest folder for that device from *before* the change (the name has the date and time).
2. Copy the file(s) you changed from that folder back onto the device — usually just `morse_map.py`, `config.py` or the secrets file. The device restarts by itself.
3. If the device won't start at all, copy **all** the `.py` files from the backup, then unplug and replug it.

There is no need to delete the broken version first — copying over it is enough.

---

6. More than one device

Why would you want two?

One AeroMorse is all you need to use it. A second one is worth having **if the AeroMorse is your only way to use the computer** — as it is for the project's author — for two reasons:

- **A spare.** If the device you rely on fails, is damaged, or is left unable to start by a bad change, the second one is plugged in and you are working again in a minute, instead of being without the computer until someone repairs or replaces it.
- **A safe place to try changes.** A new firmware version, a setting you are unsure about, or a reworked `morse_map.py` goes on the second device first. If it misbehaves, the device you depend on was never

touched. Only when it works there does it go on the daily one. Every AeroMorse release is tested this way before it is published.

A second device costs about as much as the first (roughly $30 for the recommended board, sensor and cable — an enclosure is extra; Build Guide Appendix H). It does not need its own speaker or wireless display.

How it works in practice:

- **Both can stay plugged into the computer at once.** Each shows up under its own name (below). The one with the air tube — or the switches — attached is the one in use; the other simply sits idle.
- **There is one air tube** (or one pair of switches), and moving it from one device to the other usually needs a helper. So plan it: make the change on the spare ahead of time, and have the tube moved once, when someone is with you, to try it.
- **Keep the spare ready to take over**: the same `morse_map.py` and secrets as your daily device, and sensor settings in its `config.py` that you have actually tried with it — two sensors rarely want exactly the same thresholds.
- **Back both up**, and label them so backups are never mixed up.

Keeping two devices straight

- Give each device its own label file (`AeroMorse-Green.txt` , `AeroMorse-Blue.txt`); backups and AeroMorse Secrets then show which is which.
- Give each its own **`DEVICE_NAME`** in `config.py` (e.g. `"AeroMorse Green"`), so the computer lists them by name instead of two identical "Feather ESP32-S3 Reverse TFT" entries. It applies after an unplug/replug. (If you leave it as the default "AeroMorse", the label file is used instead. A wireless display board has no `config.py` ; give it a label file such as `AeroMorse-Display.txt` and it shows as "AeroMorse Display".)
- A **wireless display** keeps its own settings in **`receiver_config.py`** on the display board (channel, brightness, rotation, timeouts). Its `ESPNOW_CHANNEL` must match the main device's `config.py` . Back it up and edit it the same way as the main device's files.
- `code.py` and `boot.py` should be the **same version** on every device. `morse_map.py` can be shared if you want the same patterns everywhere. `config.py` usually differs.
- One `macro_secrets.enc` (and one PIN) can go on every device — *Copy to device* asks which one.
- The four files carry a version line near the top (`AeroMorse code.py — version 1.30 …`); keep all four at the same version.

---

7. Starting programs with a Morse code (AutoHotkey)

AeroMorse types keys; it cannot start a program, or put the pointer at a place on the screen, by itself. **AutoHotkey** is a free Windows program that fills the gap: it watches for a key combination — say Ctrl+Alt+D — and does whatever a small script tells it to. Give that key combination a Morse code, and one code does the job.

The project supplies the script ready to use, `AeroMorse.ahk`, with three hotkeys, and the supplied `morse_map.py` already has a code for each:

Hotkey	What it does	Morse code	Group
Ctrl+Alt+D	**AeroMorse Display** window (Build Guide §5) on / off	`.-.-.-`	0 — works from every group
Ctrl+Alt+H	**AeroMorse Help** — your cheat sheet with a tab per group — on / off	`..-..-`	0 — works from every group
Ctrl+Alt+Home	Mouse pointer to the middle of the screen	`.-.`	2 — Mouse

Setting it up takes three small steps, once. AutoHotkey is Windows-only. Until the script is running the three codes do nothing useful — they send the key combination and nothing answers it.

7.1 Get AutoHotkey

Download **AutoHotkey v2** from **https://www.autohotkey.com** and run the installer (the defaults are fine). It is free and open-source.

7.2 Put the script in your work folder

Copy `AeroMorse.ahk` from the project into your **AeroMorse work folder** — `Documents\AeroMorse`, the folder set up in §1 — next to `AeroMorse Display.exe` and `AeroMorse Help.html`. The script looks for those two files in its own folder, so **there is nothing in it to edit**.

Double-click `AeroMorse.ahk` to start it. A green **H** icon appears near the clock and the three hotkeys work. If one of the two files is not in the folder, the hotkey shows a short note saying so, which closes by itself.

Keeping the script in the work folder puts everything for your AeroMorse in one place. It lives on the computer, not on the device, so **a backup of the device does not include it** — when you back up, copy the work folder too.

7.3 Make it start with Windows

Press **Win+R**, type `shell:startup` and press Enter — a folder opens. Put a **shortcut** to `AeroMorse.ahk` in that folder — a shortcut, not the script itself, which stays in your work folder (right-click the file → *Show more options* → *Create shortcut*, then move the shortcut there). From the next sign-in the hotkeys are always ready.

7.4 The three hotkeys

Ctrl+Alt+D — AeroMorse Display on / off. Opens the window if it is closed and closes it if it is open. It comes back at the size, place and see-through level you left it.

Ctrl+Alt+H — AeroMorse Help on / off. `AeroMorse Help.html` is the cheat sheet made for looking things up while you work: the legend stays at the top with one **tab per group** under it, and only the group you pick is shown — no scrolling through pages.

- Change group: click a tab, press the group's number (`0` – `9`), or use the left / right arrow keys.
- Change size: the **+** and **−** buttons in the top right corner, or the `+` and `-` keys. Only the page grows; the buttons stay where they are.
- It remembers the tab and the size you left it at, and fits the tabs and the columns to the width of the window.

It opens in a plain Microsoft Edge window of its own, with no browser tabs or address bar; the same hotkey closes it. The file is made next to `AeroMorse Cheat Sheet.pdf` every time the cheat sheet is built (*Build my cheat sheet.bat*, or `python build_pdfs.py --folder` with your work folder), from **your** `morse_map.py`, so it always matches your map. The copy that comes with the project shows the supplied map.

Ctrl+Alt+Home — pointer to the middle of the screen. For when the pointer is lost in a corner or far from where you want it. The device cannot do this by itself — a USB mouse only says "move a little this way" and never knows where the pointer is — but AutoHotkey can. With more than one monitor it goes to the middle of the main one.

Finding the pointer needs no AutoHotkey: the Mouse-group code `..-.` taps Ctrl, and Windows draws circles round the pointer if *Show location of pointer when I press the CTRL key* is ticked (Control Panel → Mouse → Pointer Options).

All three hotkeys were tried on the author's computer (Windows 11, Microsoft Edge), sent from an AeroMorse.

7.5 The Morse codes

These lines are already in the supplied `morse_map.py`. To use a different code, change the pattern on the line; `morse_map_report.txt` lists the free ones. A code in **Group 0** works from every group.

```
g0[6][0b010101] = Keycode.CONTROL, Keycode.ALT, Keycode.D      # .-.-.-  AeroMorse Display on / off
g0[6][0b001001] = Keycode.CONTROL, Keycode.ALT, Keycode.H      # ..-..-  AeroMorse Help on / off
g2[3][0b010]    = Keycode.CONTROL, Keycode.ALT, Keycode.HOME   # .-.     centre pointer
```

Changing the map is an ordinary edit: back up, edit the work copy, run the validator, and copy the file to the device only after it says PASS (§3).

7.6 Your own hotkeys, and changing the script

The same idea works for any program. Open `AeroMorse.ahk` in Notepad and add a line at the end; this one opens Notepad with Ctrl+Alt+N:

```
^!n:: Run("notepad.exe")
```

In a hotkey `^` means Ctrl, `!` means Alt and the letter is the key — change the letter if that combination is already used by something else on your computer. Then give the new combination a Morse code, as in the lines above.

After editing the script, right-click the green **H** icon near the clock and choose *Reload Script* — the change takes effect at once.

If a hotkey does nothing: check the green **H** icon is there (the script is running), that `AeroMorse.ahk` is in the same folder as the program or page it opens, and that no other program already uses the same key combination.

When the project's `AeroMorse.ahk` is updated, hotkeys you added yourself are not carried over — keep a note of them, or keep them in a second script of your own.

(The same instructions are in the Build Guide, Appendix G.)

---

8. One-page checklist

```
[ ] 1  Back up my AeroMorse                (dated backup made)
[ ] 2  Work copy refreshed from the device
[ ] 3  Free pattern found (analyzer report)
[ ] 4  Edited in Notepad++ - saved as UTF-8
[ ] 5  Analyzer re-run - no new duplicates / Group 0 clashes
[ ] 6  New secret keys? -> AeroMorse Secrets -> Save (encrypt)
[ ] 7  Check my AeroMorse files -> PASS
[ ] 8  Copied changed file(s) to the device - no sip/puff for 5 s
[ ] 9  Tested on the device
[ ] 10 Cheat sheet updated
```